

Упражнение 3:
Сериализация на обекти. Интерфейси „Serializable“ и „Externalizable“. Запис и четене на обект от файл.

инж. Лилия Андонова
ТУ – София
ФКСТ

Упражнението днес:

- Сериализация на обекти.
- Интерфейси „Serializable“ и „Externalizable“.
- Запис и четене на обект от файл.

Сериализация и десериализация

- **Сериализацията** е процесът на конвертиране на обект в поток от байтове, който може да бъде записан във файл, предаван през мрежа или запазен в база данни. Този процес позволява да се съхраняват състояния на обекти, така че те да могат да бъдат възстановени по-късно (десериализация).
- **Сериализацията се използва, когато:**
- Нужно е да се запази състоянието на обекти (например при кеширане).
- Обекти трябва да бъдат изпратени през мрежата (например при използване на RMI - Remote Method Invocation).
- Работим с файлови системи и бази данни, за да съхраняваме обекти.
- **Десериализацията** е процесът на възстановяване на обект от поток от байтове, получен чрез сериализация. Това означава, че можете да четете състояния на обекти от файл или мрежа и да възстановите обектите в паметта.

Интерфейс „Serializable“

- **Интерфейсът Serializable** е маркерен интерфейс (не съдържа методи), който позволява на класа да бъде сериализиран. Ако класът имплементира Serializable, Java автоматично ще сериализира неговите полета, включително обекти на други класове, които също имплементират Serializable.
- *Пример в допълнителния файл*

Интерфейс „Externalizable“

Интерфейсът Externalizable е разширение на Serializable, но предоставя повече контрол върху процеса на сериализация и десериализация. Класовете, които имплементират този интерфейс, трябва да дефинират два метода:

- writeExternal(ObjectOutput out) - записва състоянието на обекта в поток.
- readExternal(ObjectInput in) - възстановява състоянието на обекта от поток.

Пример в допълнителния файл

Работа с файлове

- В Java работата с файлове е основна част от взаимодействието на програмите със системата.
- Файловете могат да съдържат всякакъв тип данни: текст, изображения, видео или други структури от данни.
- Java предоставя библиотека за работа с файлове, която включва класове за четене и писане на данни във файлове.
- Основни класове за работа с файлове:
- java.io – съдържа класове за работа с потоци от байтове и символи, например File, FileReader, FileWriter, BufferedReader, BufferedWriter, PrintWriter и т.н.
- java.nio (New Input/Output) – по-нова библиотека, която предлага нови класове за работа с файлове, като Path, Files, FileSystems и други.

Обработка на грешки

За да отворим даден файл, да осъществим достъп до клавиатурата или конзолата, или да използваме друго входно/изходно устройство, се нуждаем от интерфейс, който да управлява този процес. Чрез този интерфейс улавяме потока от данни, който преминава към или от устройството, и извършваме четене или писане. При работа с външни ресурси обаче е възможно да възникнат грешки. Важно е тези грешки да не доведат до внезапно прекъсване на програмата. Вместо това трябва да бъдат уловени и обработени по подходящ начин, като например да се покаже на потребителя съобщение за грешка или да се предприемат алтернативни действия.

IOException – възниква, когато възникнат проблеми с операциите върху файлове.

SecurityException е изключение, което се хвърля, когато възникне проблем, свързан със сигурността.

FileNotFoundException, NoSuchFileException – възниква, когато файлът не може да бъде намерен.

За да се обработват тези грешки, можем да използваме try-catch блокове.

Пример в допълнителния файл

Създаване на файл

Един от начините за създаване на файл в Java е чрез използване на класа **File**, който предоставя множество методи за работа и достъп до файлове.

Един от тези методи, който ни интересува, е **createNewFile()**. Този метод създава нов празен файл, ако файлът с даденото име все още не съществува, и връща true, когато файлът е успешно създаден.

Пример в допълнителния файл

Писане във файлове

За да пишем във файл трябва да се отвори изходен поток, след което да се инициализира „писец“ до файла и да се запише желаното в него чрез методите, които писецът предлага.

Байтовите потоци се използват, за да трансферират байтове от данни(8-bit bytes). Най-често се използват FileInputStream и FileOutputStream. Обикновено, за да се инициализира такъв поток, му е нужно име на файл. Те позволяват директно четене и писане във файла.

Основни методи на FileInputStream:

int read(): Чете един байт от файла и връща неговото значение (като цяло число от 0 до 255). Ако достигне края на файла, връща -1.

int read(byte[] b): Чете до b.length байта в подадения масив b. Връща броя на прочетените байтове или -1, ако достигне края на файла.

skip(long n): Пропуска зададен брой байтове в потока.

void close(): Затваря потока и освобождава всички системни ресурси, свързани с този поток.

Основни методи на FileOutputStream:

void write(int b): Записва един байт във файла.

void write(byte[] b): Записва всички байтове от подадения масив b във файла.

flush(): Принудително изписва всички натрупани байтове в потока към файла.

void close(): Затваря потока и освобождава всички системни ресурси, свързани с него.

Знаковите потоци (Character Streams) се използват за предаване на 16-битови данни във формат Unicode. Най-популярните сред тях са FileReader и FileWriter.

Класът **FileReader** е част от Java I/O библиотеката и се използва за четене на текстови файлове. Той е предназначен за четене на файлове в текстов формат. Това го прави подходящ за работа с текстови файлове.

Основни методи:

read(): Чете следващия символ от файла и връща неговата стойност като цяло число (Unicode стойността на символа). Когато достигнете края на файла, методът връща -1.

read(char[] cbuf): Чете множество символи наведнъж и ги записва в масив от символи. Връща броя на прочетените символи.

close(): Затваря потока и освобождава ресурсите, свързани с него.

Класът **FileWriter** също е част от Java I/O библиотеката и е предназначен за писане на текстови файлове.

Основни методи:

write(int c): Записва един символ в потока.

write(char[] cbuf): Записва масив от символи в потока.

write(String str): Записва цяла низова стойност в потока.

flush(): Принудително изписва всички натрупани данни в потока.

close(): Затваря потока и освобождава ресурсите, свързани с него.

Буферирани и небуферирани потоци

В Java, при работа с файлове и потоци, има два основни типа потоци, които се използват за четене и писане на данни: небуферирани и буферирани потоци. Разликата между тях е свързана с начина, по който се извършва четенето и писането на данни, и дали използват междинна памет (буфер), за да оптимизират производителността.

Небуферираните потоци (Unbuffered Streams) четат или записват данни директно, като правят операциите на всяко четене или запис поотделно, без използване на междинна памет. Това може да доведе до по-бавни операции, защото всяко четене или писане във файловата система е сравнително бавно. (FileInputStream, FileOutputStream)

Буферираните потоци (Buffered Streams) използват междинна памет (буфер), която прави четенето и писането на данни по-ефективно. Когато четете или пишете, данните първо се зареждат в буфера, а след това се записват в/извеждат от файла на по-големи парчета. Това значително ускорява процеса, тъй като минимизира броя на операциите с диска (което е много по-бавно от обработката на данни в паметта). (BufferedInputStream, BufferedOutputStream, BufferedReader и BufferedWriter)

BufferedReader и BufferedWriter

BufferedReader и BufferedWriter са класове в Java, които се използват за ефективно четене и писане на данни в текстови файлове. Те се намират в пакета java.io и предоставят буферирани операции, които значително увеличават производителността при работа с файлове, като намаляват броя на операциите за четене и писане.

BufferedReader се използва за четене на текстови данни от потоци, като използва вътрешен буфер, който временно съхранява данни. Това позволява да се четат по-големи блокове данни наведнъж.

Основни методи:

read(): Чете един символ от потока. Връща -1, ако няма повече данни.

readLine(): Чете един ред от текста.

close(): Затваря потока и освобождава ресурсите.

BufferedWriter се използва за запис на текстови данни в потоци, като също използва вътрешен буфер, за да оптимизира записите. Данните се натрупват в буфера и когато той е пълен или когато извикате метода flush(), те се записват в изходния файл или поток.

Основни методи:

write(String s): Записва низ в потока.

newLine(): Записва нов ред

flush(): Изпразва буфера и записва всички натрупани данни в потока.

close(): Затваря потока и освобождава ресурсите.

Какво научих днес:

- Сериализация на обекти.
- Интерфейси „Serializable“ и „Externalizable“.
- Запис и четене на обект от файл.